

Learning Blackjack with First-Visit Monte Carlo Reinforcement Learning

Cesar Gonzales
Johns Hopkins University
Baltimore, MD, United States
February 24, 2026

I. INTRODUCTION

A. Hypothesis

The game Blackjack is designed to favor the dealer in that a player is expected to have negative overall profits over the course of many games [1]. Edward Thorp claimed in [1] that a player is expected to lose 2% or more of their wagered money over time using sub-optimal strategies. He came up with a “basic” strategy for playing Blackjack as a way of reducing the dealer’s edge to about 0.5% at worst and possibly a 0.1% advantage over the dealer when playing with favorable rules. Like Sutton and Barto in [2], we’ll refer to a slight variant of this “basic” strategy as the optimal strategy for playing our version of Blackjack, which is discussed more below. We propose that a Monte Carlo-based Reinforcement Learning (MC-based RL) agent can learn to play Blackjack while achieving an average return score that is at least 80% of the average return score achieved by an agent using the optimal strategy. Furthermore, we believe that a MC-based RL agent is capable of achieving a significantly higher performance than an RL agent using a policy of uniformly random actions.

II. METHODS

A. Environment

Our version of Blackjack consists of states that are encoded as 3-digit integers. For each state encoding, the ones digit represents the player point total $p - 12$, the tens digit represents the dealer’s face-up card $f - 1$, and the hundreds digit represents whether a usable ace card is present (i.e., either 0 or 1). Overall, there are 200 decision states (encoded from 0 to 199), 1 unused state (200), and three terminal states. Terminal state 201 occurs when the agent loses, terminal state 202 represents a tie, and terminal state 203 occurs when the agent wins. When a game is not immediately decided after the cards are dealt, the agent can either ‘stick’ to remain with their current hand or ‘hit’ to have more cards dealt. For simplicity, other actions like ‘split’ and ‘double’ are not included in this version of Blackjack. For every game, the agent receives a deterministic reward. If the agent loses (i.e., state 201), then they receive a reward of $R = -1$. If they tie (i.e., state 202), then they receive a reward of $R = 0$. If they win (i.e., state 203), then they receive a reward of $R = 1$.

B. Agents

We implemented three different types of agents for learning Blackjack, two of which are considered learning agents, and one is rule-based. Each type of agent uses a different policy (i.e., strategy) for playing the game. The first learning agent type is a First-Visit MC-based RL agent that uses an ϵ -greedy soft decision policy for action selection during training. By default, we used an epsilon of $\epsilon = 0.2$. Note that our game can be modeled as an episodic Markov Decision Process (MDP) with a finite horizon, and so no delayed reward discounting is required for algorithm convergence. Thus, we used a discounting factor of $\gamma = 1.0$. Our second type of learning agent is a random agent based on our first agent type, but with epsilon set to $\epsilon = 1.0$. Consequently, action selection is always performed randomly for every training episode. This type of agent essentially learns a random policy, which may or may not be optimal. Finally, our third type of agent was implemented using the rule-based optimal strategy defined by [2] and outlined below.

Optimal Strategy for playing Blackjack (as defined by Sutton and Barto [2])

- If there is a usable ace:
 - If the dealer card is valued at 1, 9, or 10:
 - * Hit if your sum is ≤ 18 , else stick
 - Otherwise:
 - * Hit if your sum is ≤ 17 , else stick
- If there is no usable ace:
 - If the dealer card is valued at 1 or 7 through 10:
 - * Hit if your sum is ≤ 16 , else stick
 - If the dealer card is valued at 2 or 3:
 - * Hit if your sum is ≤ 12 , else stick
 - Otherwise:
 - * Hit if your sum is ≤ 11 , else stick.

This optimal strategy differs from the “basic” strategy defined in [1] only when there is a usable ace card present, and the dealer’s face up card is valued at one. In this case, [1] recommends to ‘hit’ if the player’s card sum is less than or equal to 17, otherwise ‘stick’.

C. Agent Learning

Note that the rule-based agent is meant to always follow the same policy, so no learning was performed for this type

of agent. To get an accurate estimate of performance for our other two agents, we trained and evaluated 100 MC-based RL agents and 100 random agents. Each of these agents played 5000 episodes of Blackjack with learning enabled followed by 100 episodes with learning disabled. We then ran a single rule-based agent that played just 100 episodes with learning disabled. With this approach, each agent type could be evaluated over the 100 episodes played with learning disabled.

D. Data Collection and Processing

Our main metric for evaluating each agent's performance is the average return per episode. To evaluate our learning agents (i.e., the MC-based RL and random agent), we first computed each episode's average return across all the agents of that type. That is, we computed the following sum for both types of learning agents:

$$G_i = \frac{1}{N} \sum_{j=1}^N G_{i,j} \quad 1 \leq i \leq 5100$$

, where $N=100$ and is the number of agents for each type. Averaging over the agents allowed us to mitigate the variability induced by differences between the agents' decision policies as the agents learn to play Blackjack and the different sets of cards each agent experiences.

III. RESULTS

A. Learning Curves

To acquire insight into the learning agents' performance while learning was enabled, we plotted learning curves, as shown in Fig. 1 and Fig. 2. To minimize data noise, we applied a simple smoothing (i.e., averaging) function defined by the following equation:

$$\bar{G}_i = \frac{1}{w} \sum_{k=i-\lfloor w/2 \rfloor}^{i+\lfloor w/2 \rfloor} G_k \quad 1 \leq i \leq 5000$$

, where w is the window size. We initially tried a window size of $w = 21$, which turned out to be insufficient. We then increased the window size to $w = 201$ for our final plots shown in Fig. 1 and Fig. 2. Note the apparent contrast in learning experiences for each of our learning agent types in these plots. Clearly, the average performance of the MC-based RL agents is gradually improving with more played episodes. In contrast, the average return of the random agents is lower and does not appear to definitively increase (nor decrease) as more episodes are played. This makes sense because the MC-based RL agents are designed to converge to an optimal policy as the number of episodes approaches infinity. The random agents are merely choosing random actions for each episode and thus never learn to play better in Blackjack.

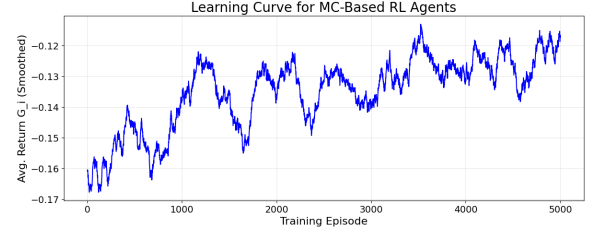


Fig. 1: The average returns across 100 MC-based RL agents over 5000 training episodes.

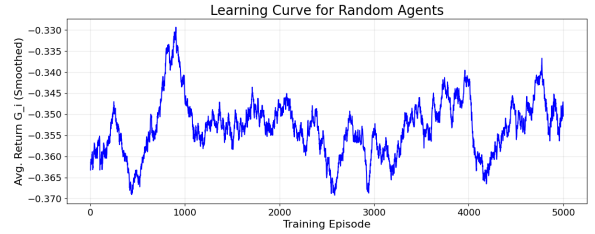


Fig. 2: The average returns across 100 random agents over 5000 training episodes.

B. Agent Evaluation and Comparison

As mentioned above, each trained agent ran for 100 episodes with learning disabled. The single rule-based agent, which required no learning, also ran for 100 episodes. We then computed the total average return per episode for each agent type. Following the computation of the average return G_i across the 100 agents for each learning agent type, we then computed the total average return per episode as follows:

$$\bar{G} = \frac{1}{100} \sum_{i=5001}^{5100} G_i$$

For the rule-based agent, we simply computed the average return per episode as follows:

$$\bar{G} = \frac{1}{100} \sum_{i=5001}^{5100} G_i$$

We then compared the results as shown in table I. As you can see, the rule-based agent performs the best, which makes sense since it uses our definition of the optimal policy for our version of Blackjack. Note that the rule-based agent's average return is nearly a factor of 10 worse than the performance of -0.5% that is claimed by Thorp in [1]. That said, the standard deviation for this agent is relatively higher than the others. This is probably because we only trained one instance of the rule-based agent, whereas we trained 100 instances and averaged the performances for the other types. On average, the MC-based RL agents performed worse than the rule-based agent but better than the random agents, which is encouraging considering we only trained the learning agents for 5000 episodes. Surprisingly, the random agent's average performance is only slightly worse than that of the MC-based

Agent Type	G_{avg}	Std Deviation
MC-based RL	-0.083	0.092
Random	-0.093	0.104
Rule-based	-0.03	0.964
MC-based RL (epsilon decay)	-0.073	0.093
MC-based RL (50,000)	-0.045	-0.089

TABLE I: The average return for each agent type over 100 episodes with learning disabled. The second row from the bottom corresponds to the results for the MC-based agents using decayed-epsilon-greedy action selection. The last row corresponds to the results for the MC-based agents trained over 50,000 episodes each.

agents. From the learning curves, one might guess that the MC-based RL agents would largely outperform the random agents. However, this result is likely due to two factors. One, during evaluation learning is disabled, which means ϵ -greedy action selection is no longer performed for any of the agents. That is, even the random agents have converged to some set of policies at this point. Instead of continuing to play randomly, the random agents use these converged policies to select the "most optimal" actions during the 100 evaluation episodes. The other factor is that the outcome of each episode is likely largely determined by the chance component of Blackjack, which can sway the results in either direction.

C. Decayed-epsilon-greedy Action Selection

We experimented with decaying the epsilon value while training the learning agents. This was only performed for the MC-based RL agents since the random agents by definition always use an epsilon of $\epsilon = 1.0$. For the MC-based agents, we linearly decayed the value of epsilon over the first 2,000 training episodes from $\epsilon = 0.7$ to $\epsilon = 0.02$. The corresponding learning curve is shown as Fig. 3 in the appendix. You can see that the average return per episode during training converges much faster and to a higher value. For evaluation, the average return per episode with learning disabled is shown in the second row from the bottom in table I. Clearly, decayed-epsilon-greedy action selection improves the average performance of the MC-based agents.

D. Training for More Episodes

We decided to observe the effect of increasing the duration of the training of the learning agents, specifically up to 50,000 episodes. As expected, the performance of the random agents did not improve (so we did not report it). The last row of table I shows the results for the MC-based RL agents. You can see that training for 10 times more episodes nearly halves the performance (i.e., an improvement) for this type of agent. We believe that with more learning the average performance of the MC-based RL agents will continue to improve and converge toward the optimal value of -0.5% .

IV. CONCLUSION

Even when playing Blackjack optimally, one's expected profits are negative over time. On the other hand, our results show there is still a skill aspect of playing the game, and a player with less skill (e.g., our random agent) will yield less profits on average. Most importantly, our results demonstrate that a MC-based RL agent can learn to attain this skill

component of Blackjack while approaching the performance level of a player that plays optimally.

REFERENCES

- [1] Thorp, Edward O., "Beat the dealer: A winning strategy for the game of twenty-one", Vintage, 2016.
- [2] Sutton, R., Barto, A., "Reinforcement Learning: An Introduction", 2nd ed., The MIT Press, 2018

APPENDIX

A. Additional Plots

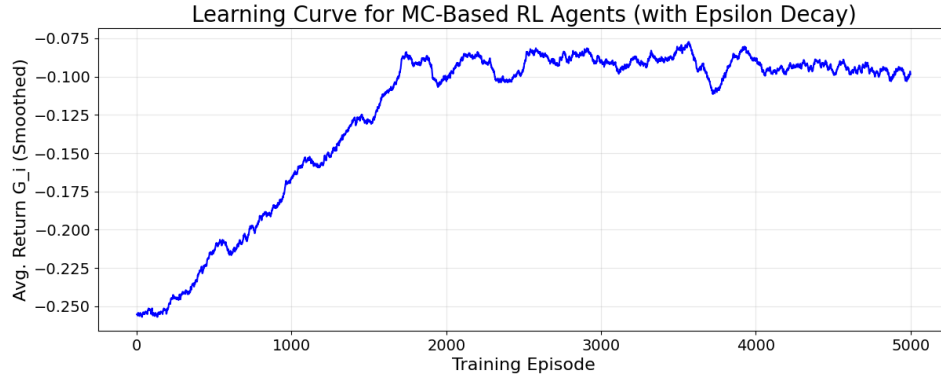


Fig. 3: The average returns across 100 MC-based RL agents using decay-epsilon-greedy action selection.

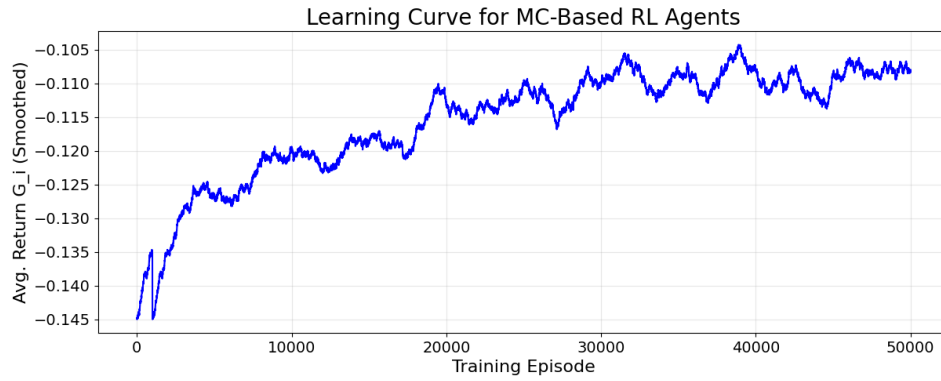


Fig. 4: The average returns across 100 MC-based RL agents over 50,000 training episodes.

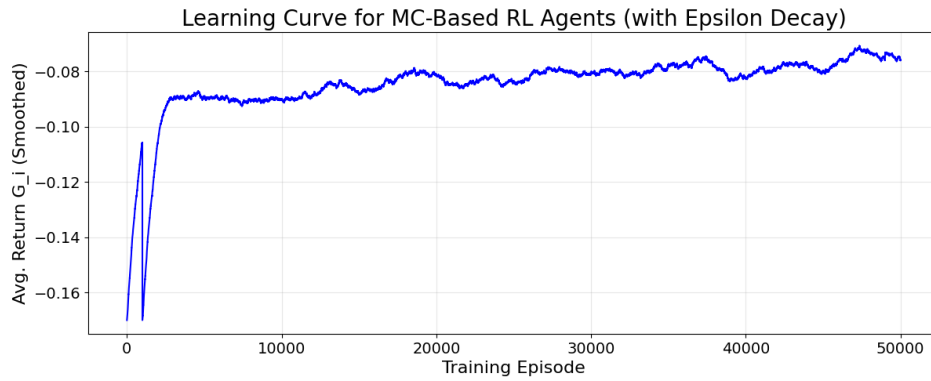


Fig. 5: The average returns across 100 MC-based RL agents over 50,000 training episodes with decay-epsilon-greedy action selection.